

LUKE SCHREIBER

Engineering Portfolio · Test, Instrumentation & Digital Hardware

lukeschreiber.com
github.com/LukeSchreiber
Schreiber.212@osu.edu
Columbus, OH

Electrical & Computer Engineering at The Ohio State University, B.S. expected **May 2029**. I spent Summer 2026 as an Electrical Test Engineer Intern with HX5 at **NASA Glenn Research Center**, in the Space Environments Test Branch — working on thermal-vacuum facility electrical systems and leading the electrical side of a five-person student design project. The work in this portfolio is mostly instrumentation: getting a physical quantity into a wire, into a number, and into a decision.

CONTENTS

01

Solid Rocket Motor Test Stand · p.2–4

02

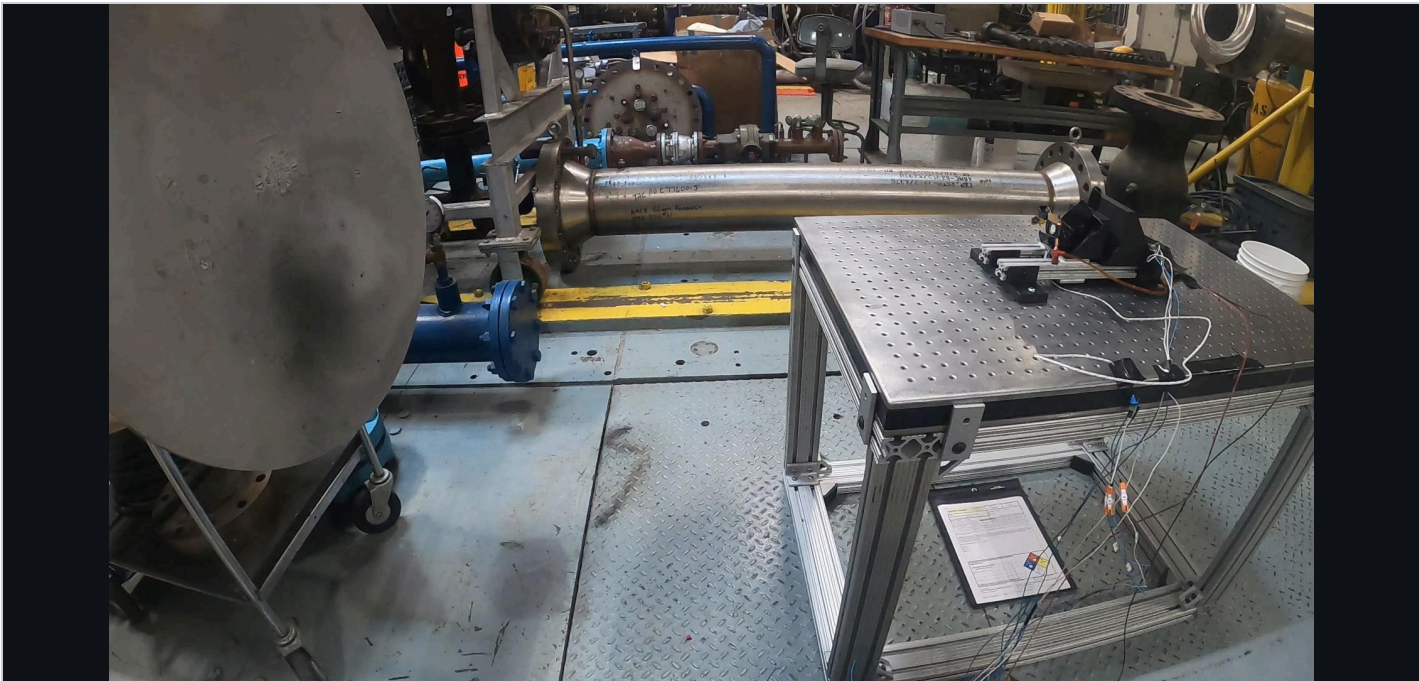
Space Environments Test Branch · p.5

03

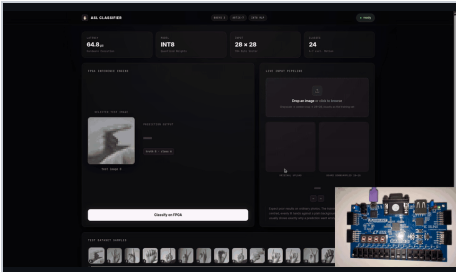
FPGA ASL Classifier · p.6

04

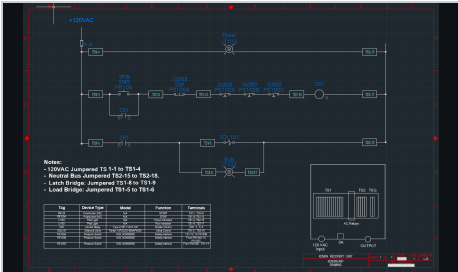
VeloLift Velocity Tracker · p.7



COMPLETED HORIZONTAL THRUST STAND INSTALLED FOR LIVE-FIRE TESTING – NASA GLENN, SUMMER 2026



01 · FPGA inference engine, 64.8 μ s latency



02 · XENON CART CONTROL SCHEMATIC



03 · ARM / IGNITION CONTROL BOX

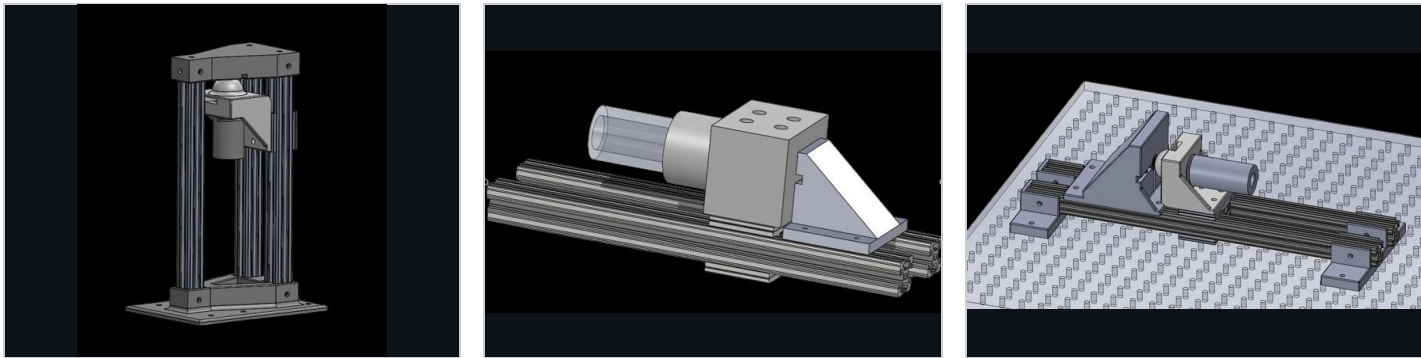
HARDWARE Verilog · FPGA · embedded C/C++ · PLCs · load cells & signal conditioning	TOOLS Vivado · AutoCAD Electrical · SolidWorks FEA · Python · PyTorch · Git
--	---

Solid Rocket Motor Static Test Stand

MY ROLE	TEAM	TIMELINE	STACK
Electrical lead — instrumentation, control logic, firmware, data analysis	5 interns across EE, ME, AE, CS	Summer 2026 · 10 weeks	Arduino C++ · HX711 · thermocouple · SD · SolidWorks · Python

The challenge: build a stand that measures real thrust and exhaust temperature from commercial model rocket motors, then use the measured curves to size a staged vehicle for a defined mission. Published manufacturer thrust curves are the industry reference — we wanted to know how closely a low-cost instrumented stand could reproduce them, and where the error actually comes from.

DESIGN ITERATION



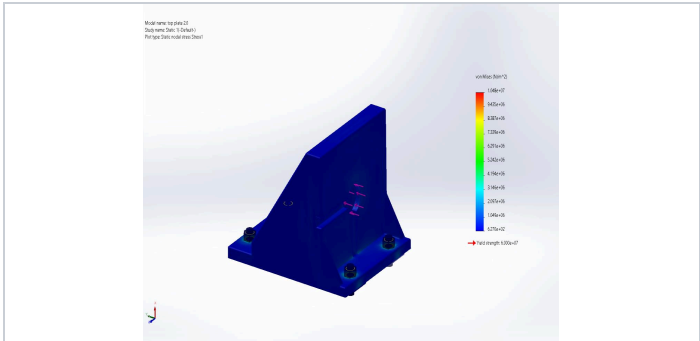
V1 – VERTICAL, 80/20 FRAME

V2 – HORIZONTAL SLED CONCEPT

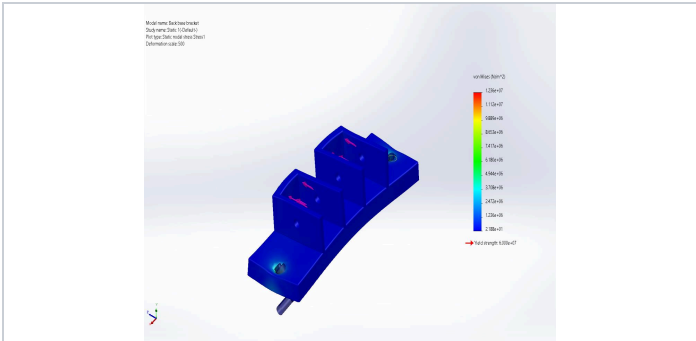
V3 – FINAL, OPTICAL-TABLE MOUNTED

Why the design changed. The vertical stand loaded the motor along gravity, which was the physically cleaner arrangement — but once printed and assembled it weighed **≈11 oz against a 4 oz target**. That dead mass sat directly in the load path and swamped the thrust signal from the smaller motors. Rather than chase weight out of the structure, we rotated the whole design horizontal, which removes stand weight from the measurement entirely. Several 3D-printed parts from the vertical prototype carried straight over.

STRUCTURAL VALIDATION



TOP PLATE – MAX STRESS 1.04E7 N/M², MIN FOS 5.7



BACK BASE BRACKET – MAX STRESS 1.236E7 N/M², MIN FOS 4.9

ORIENTATION TRADE STUDY

VERTICAL – ARGUMENTS FOR

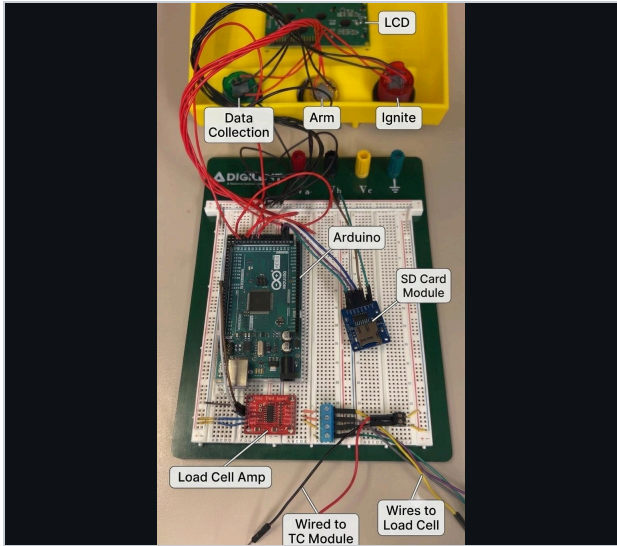
- Thrust axis aligned with gravity; reduces side loads and bending moments on the mounts.
- Black powder combustion behaves more like a real launch; hot gas doesn't pool to one side.
- Simpler alignment of the motor to the load path.

VERTICAL – ARGUMENTS AGAINST

- Exhaust reflects back up into the wiring and printed parts.
- Stand mass sits in the measurement and has to be subtracted out.
- Engine install and wiring access are both worse; tipping, vibration and twisting all in play.

Instrumentation, Control Logic & Safety

SAMPLE RATE 80 Hz load cell + thermocouple	LOAD CELL 12.5 kg HX711 amplifier	IGNITION WINDOW 4 s firmware-limited	ARM SEQUENCE 2-button isolated firing circuit
---	--	---	--

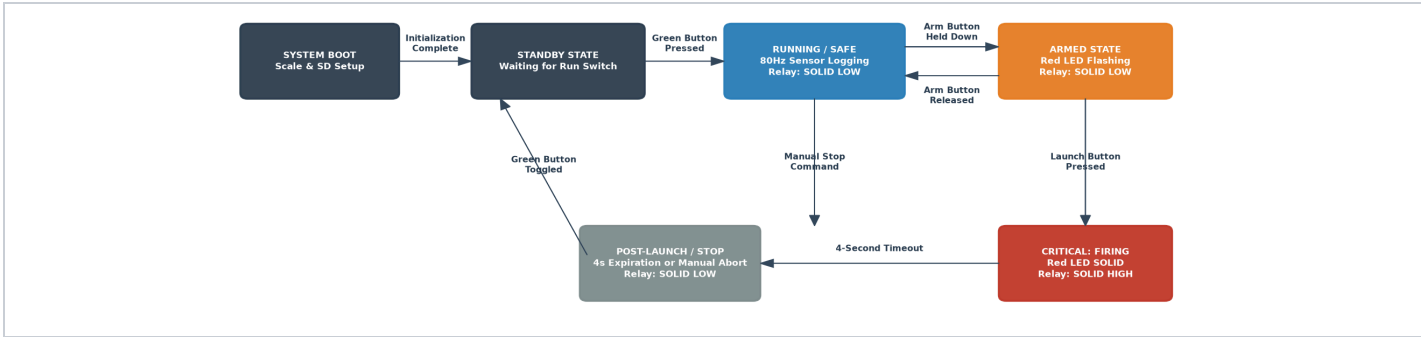


BENCH BUILD – ARDUINO, HX711, SD MODULE, TC INPUT

WHAT I BUILT

- Arduino C++ firmware handling **acquisition, display, logging and ignition** in a single deterministic loop — load cell and thermocouple sampled, converted to thrust and temperature, written to SD as CSV.
- **Dual-button arm/launch interlock:** the stand cannot fire unless it is armed and the launch button is pressed. Red indicator flashes while armed, goes solid on fire.
- Firing circuit **electrically isolated from the sensor circuit** so ignition current cannot corrupt the thrust trace.
- Ignition signal hard-limited to **4 seconds**, with logging continuing straight through the burn and a manual abort path from any state.
- LCD showing live thrust, temperature, elapsed time and arm/ready status for the operator at the stand.
- Soldered and assembled the wiring harnesses; documented the build and the calibration procedure.

Sampling rate trade. 10 Hz was enough to see the burn envelope but blurred the ignition spike, which is where the peak thrust lives. We moved to **80 Hz** — enough resolution to catch the transient without outrunning the HX711 or the SD write.



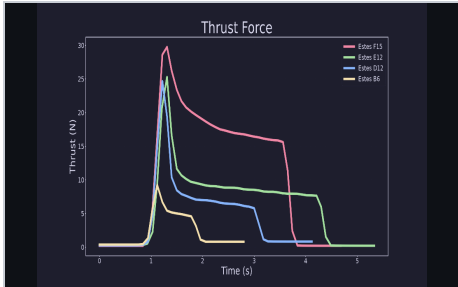
FIRMWARE STATE MACHINE – BOOT → STANDBY → RUNNING/SAFE → ARMED → FIRING → POST-LAUNCH

WHAT WENT WRONG, AND WHAT IT TAUGHT ME

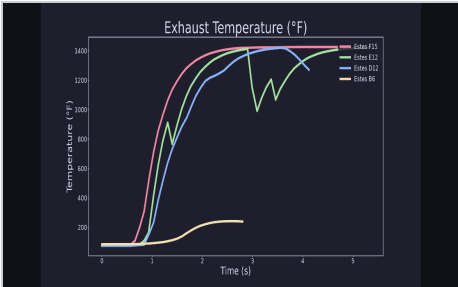
- **Lost a full run.** The load cell terminal block didn't make complete contact during the C6 test — no thrust data captured. Fix was mechanical, not electrical: strain relief and a connection check added to the pre-fire checklist.
- **Thin, short load cell leads** made every reconnection a risk. On a rebuild I'd pigtail them to a proper terminal immediately rather than working the original wire.
- **Intermittent LCD language changes** pointed at a noisy or marginal data line — a reminder that a display glitch on a test stand is a signal integrity symptom, not a cosmetic one.

Test Results & Mission Application

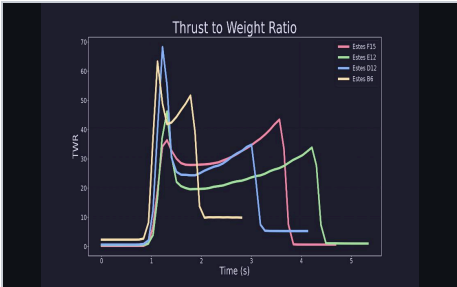
Four Estes motor classes were fired and logged. Raw CSV was cleaned and plotted through a small Python pipeline, then each measured curve was compared against the published ThrustCurve reference for that motor.



THRUST FORCE, ALL MOTORS



EXHAUST TEMPERATURE



THRUST-TO-WEIGHT RATIO

MEASURED VS PUBLISHED

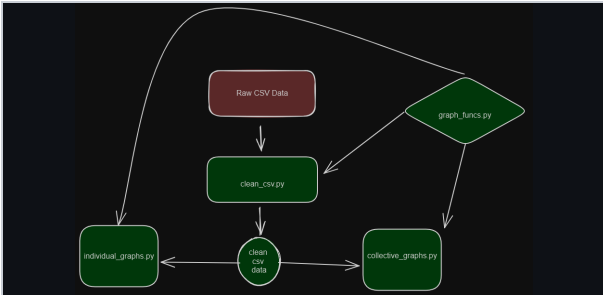
MOTOR	MAX THRUST	TOTAL IMPULSE	SPECIFIC IMPULSE	PEAK EXHAUST	Δ THRUST	Δ IMPULSE
Estes B6	9.22 N	5.00 N·s	62.03 s	241 °F	23.8%	16.3%
Estes D12	24.66 N	17.99 N·s	76.12 s	1420 °F	17.0%	7.1%
Estes E12	25.28 N	32.13 N·s	83.71 s	1421 °F	24.1%	18.1%
Estes F15	29.74 N	49.66 N·s	87.11 s	1425 °F	17.5%	0.1%

Δ MEASURED AGAINST PUBLISHED THRUSTCURVE DATA FOR EACH MOTOR

WHERE THE ERROR COMES FROM

- **Load cell hysteresis** as thrust rises and falls across the burn — the dominant term, and the reason peak thrust deviates more than total impulse.
- Sensor **non-linearity and quantization**; uncertainty in the calibration weights themselves propagates into every reading.
- **Sled friction** in the horizontal configuration, plus possible **heating of the load cell** during longer burns.
- Component degradation across repeated firings.

Impulse tracked the reference far better than peak thrust — F15 came in at **0.1% on total impulse** while still showing 17.5% on peak. Integrating over the burn averages out the transient error that a single peak sample carries.



PYTHON ANALYSIS PIPELINE — RAW CSV TO PER-MOTOR AND COLLECTIVE PLOTS

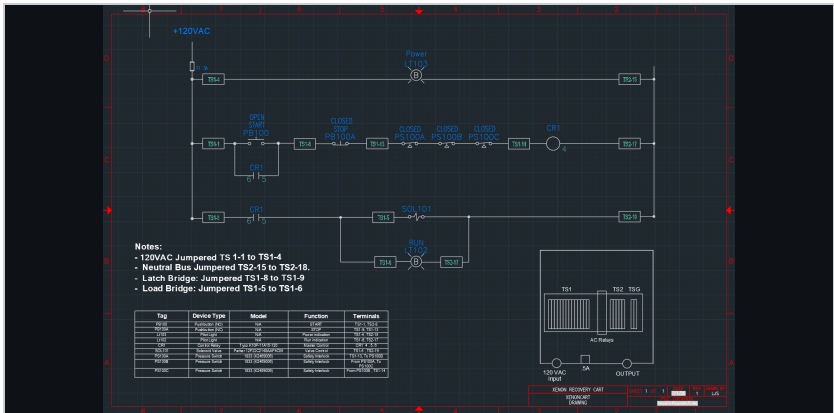
MISSION APPLICATION — STAGED VEHICLE TO HUBBLE'S ORBIT

Using the measured impulse rather than published values, the team sized a staged vehicle to reach the Hubble Space Telescope's orbit at 483 km. Vis-viva gives 7.626 km/s for the circular orbit; adding gravity and drag losses of 1.4 and 0.2 km/s puts the requirement at **9.226 km/s**. Two staged D12 motors deliver **9.333 km/s** across 963.9 g of propellant — enough margin to also cover a Hohmann transfer up to 600 km.

STAGE	MOTOR	MASS BEFORE BURN	MASS AFTER BURN	STAGE ΔV
Stage 1	Estes D12	1006.1 g	524.2 g	1.460 km/s
Stage 2	Estes D12	710.3 g	21.1 g	7.873 km/s
Total	—	—	963.9 g propellant	9.333 km/s

Space Environments Test Branch

ROLE	FACILITY	TERM	TOOLS
Electrical Test Engineer Intern	Thermal-vacuum test chambers, electric propulsion support	Summer 2026	AutoCAD Electrical · PLCs · vacuum gauges & pumps



XENON RECOVERY CART – CONTROL SCHEMATIC, REDRAWN AND STANDARDIZED

XENON RECOVERY CART

- Produced a complete **control schematic** for the cart used in electric propulsion testing — start/stop latching circuit, control relay, solenoid valve and pressure-switch interlocks off 120 VAC.
- Standardized every device tag** to TS/PB/CR/SOL convention with full terminal mapping, so the sheet can actually be used to wire and troubleshoot the panel.
- Delivered as a build package — **title block, revision table and BOM** — with extraneous views stripped so it stays focused on power and control.

PREVENTIVE MAINTENANCE PROGRAM

- Built a **ventilation controls PM plan** covering every TC010M / TC010MA turbopump controller in the branch.
- Intervals driven by **controller hour-meter runtime** — 24-hour airflow checks through 2,000-hour fan replacement and 40k/80k/120k-hour overhaul alarms.
- Replaced ad-hoc record keeping with a **unified checklist and log**.

PREVENTIVE MAINTENANCE SCHEDULE	
Track runtime using the controller Hour Meter (menu M03). Applies to every TC010M / TC010MA controller listed above.	
INTERVAL	MAINTENANCE TASK
Every 24 h	Airflow quick check: controller secured; vents/fan unobstructed; environment OK; cables secure; ALARM/FAILURE not lit; avoid rapid start/stop cycling.
Every 100 h	Status review: log temperatures (SD4) and Error History / Maintenance Calls (M07/M08).
Every 250 h	Clean vents (vacuum + dry air); inspect back-panel fan for noise, wobble, free rotation.
Every 500 h	Clean fan guards/filters and dry fully before reinstall; inspect cabling/grounding; respect bend radius.
Every 1,000 h	Cooling audit: compare internal vs ambient temperature; improve ventilation if trend increases.
Every 2,000 h	Preventive fan replacement; export maintenance/error logs; reset counters per lab policy.
40,000 h: L+1	Maintenance-time alarm Level 1: begin long-term service planning.
80,000 h: L+2	Maintenance-time alarm Level 2: prepare controller overhaul/replacement plan.
120,000 h: L+3	Maintenance-time alarm Level 3: overhaul or replace controller (clearing the alarm does not reset cumulative hours).

HOURLY-METER BASED PREVENTIVE MAINTENANCE SCHEDULE



GRANVILLE-PHILLIPS GAUGE



BUSY BEE GAUGE

GAUGE CART OPERATIONS

- Qualified as a gauge cart operator** and executed calibration test runs independently.
- Documented results and status for each unit, **clearing the branch's outstanding backlog**.
- Hands-on with roughing pumps, high-vacuum pumps, gauge types and the PLCs tying them together.



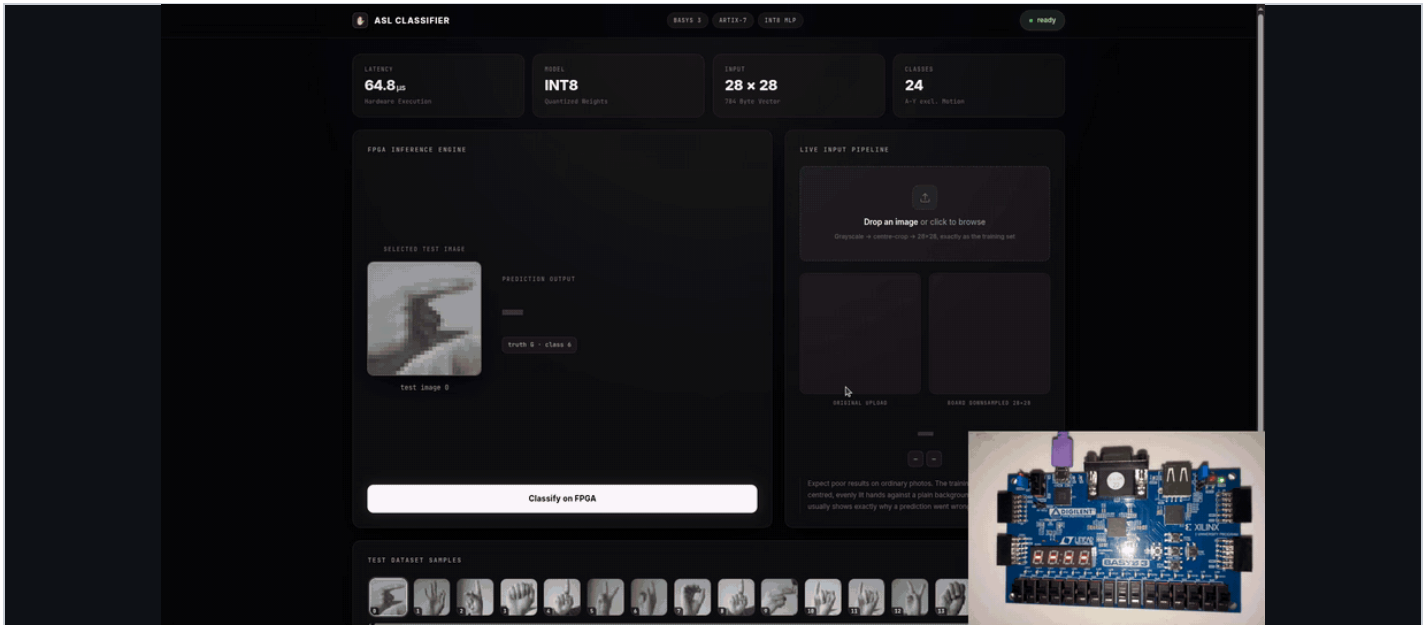
OUTSTANDING SUMMER 2026 INTERNSHIP – OFFICE OF STEM ENGAGEMENT

WHAT THE SUMMER ACTUALLY TAUGHT ME

- Tools are application layers — the engineer solves the problem; AutoCAD and the PLC are where the answer gets written down.
- Technical strength with weak communication is capped impact.
- Work upward from the simplest solution; a lot of facility problems end there.

FPGA ASL Classifier — Quantized Inference in Hardware

TARGET Digilent Basys 3 · Xilinx Artix-7	MODEL INT8 quantized MLP, trained in PyTorch	STACK Verilog · Vivado · Python · serial host UI	STATUS Working on board; RTL being rebuilt by hand
LATENCY 64.8 μ s hardware execution	PRECISION INT8 quantized weights	INPUT 28 $\times$ 28 784-byte vector	CLASSES 24 A–Z excl. motion letters



HOST DASHBOARD DRIVING THE BASYS 3 OVER SERIAL — TEST IMAGE, PREDICTION, AND LIVE UPLOAD PIPELINE

HOW IT WORKS

- An MLP is trained in **PyTorch on the ASL alphabet set**, then quantized to INT8 and mapped to a fixed-point datapath in hardware.
- The board holds the weights and runs the full forward pass on-chip; the host sends a 784-byte vector and reads back a predicted class and logit.
- A browser dashboard drives the board over serial — pick a test image or drop in a photo, which is **converted to grayscale, centre-cropped and downsampled to 28×28** to match the training distribution.
- Every prediction is checked against a **software reference model** running the same quantized weights, so a hardware/software mismatch is immediately visible.

TRAINING

Float model trained in PyTorch on the ASL alphabet set, then post-training quantized to INT8 so weights and activations fit an integer datapath.

ON THE BOARD

Weights held on-chip; the full forward pass runs in hardware and returns a class index and logit. Seven-segment display mirrors the prediction at the board.

HONEST STATE OF THE PROJECT

The first version of this RTL was written with heavy AI assistance. It runs on the board and the numbers are real, but I am rebuilding the datapath by hand — accumulator sizing, quantization scaling and the control FSM — so that I can defend every line of it. I would rather say that plainly than be caught out on it in a whiteboard session.

What I'd change. The MLP was the right first target for learning the toolchain, but the interesting hardware problem is a systolic or tiled MAC array with proper weight reuse. That's the next build.

HOST LINK

Serial transport carrying a 784-byte vector down and the result back, with the software reference model running the same weights for comparison.

VeloLift — Barbell Velocity Tracker

HARDWARE

Arduino Uno/Nano · 600 PPR rotary encoder

SOFTWARE

Python / FastAPI backend · React frontend

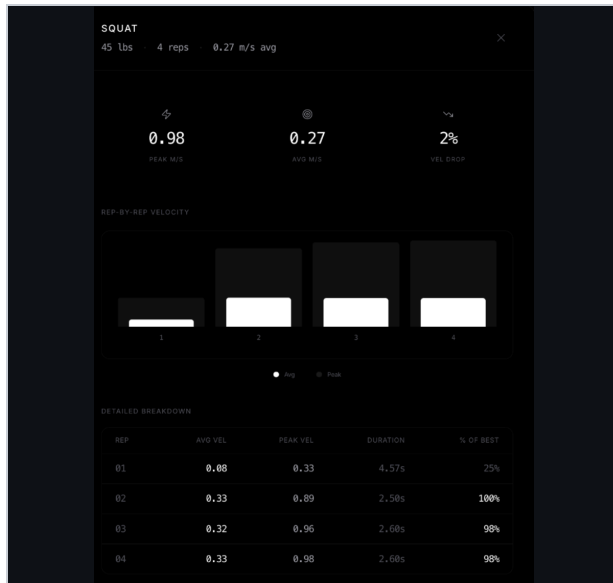
LINK

github.com/LukeSchreiber/VeloLift

STATUS

Working prototype, in use in the gym

Velocity-based training uses bar speed as a live proxy for fatigue and effort — but commercial trackers cost several hundred dollars. This is a tethered encoder measuring bar displacement directly, streaming velocity to a logger that records what actually happened on each rep.



PER-SET BREAKDOWN — PEAK, AVERAGE, AND VELOCITY DROP ACROSS REPS

SIGNAL PATH

- The encoder is tied to the bar; pin A is on a **hardware interrupt** and pin B is read inside the ISR to resolve direction, so up and down are signed rather than absolute.
- Pulses convert to distance through a measured spool circumference and **600 pulses/rev**, differentiated over a fixed 100 ms window; a deadband under 0.05 m/s keeps a shaking rack from registering as movement.
- Serial at **115200 baud** into a FastAPI service, which **streams at 50 Hz over WebSocket** to the frontend and auto-detects the Arduino port.
- Rep counting is a **direction-based state machine** on the velocity sign — no manual rep entry.
- 3D-printed enclosure in OpenSCAD; set history stored locally in IndexedDB.

Reading the data. In the set shown, rep 1 comes in low — that's the unrack and setup, not a working rep. Reps 2–4 hold 0.32–0.33 m/s average with only **2% velocity drop**, which says the load was well under the point where bar speed starts to decay.

```
volatile long pulseCount = 0;

// ISR: fires on every rising edge of channel A.
// Channel B is sampled inside the ISR to resolve direction.
void handleEncoder() {
  if (digitalRead(pinB) == HIGH) pulseCount--; // bar descending
  else                          pulseCount++; // bar ascending
}

// 100 ms window -> signed velocity in m/s
noInterrupts();
long currentPulses = pulseCount; pulseCount = 0;
interrupts();
float velocity = (currentPulses * metersPerPulse) / 0.1;
```

Encoder firmware — interrupt-driven counting with direction resolution

DESIGN DECISIONS

- Counting in an **ISR rather than polling** — at 600 PPR and useful bar speeds the pulse rate outruns any loop that also has to talk over serial.
- The pulse counter is copied out with **interrupts disabled**, then zeroed, so a pulse arriving mid-read can't be double counted or lost.
- Velocity is computed on the Arduino, not the host, so the timing base stays on the device that owns the interrupt. Sign convention puts **positive upward**, which makes the concentric phase trivially separable for rep detection.

EDUCATION

The Ohio State University

B.S. Electrical & Computer Engineering

Expected May 2029

Transfer from Lorain County CC · 79 credit hours · GPA 3.53

CONTACT

luke@schreiber.com

github.com/LukeSchreiber

Schreiber.212@osu.edu